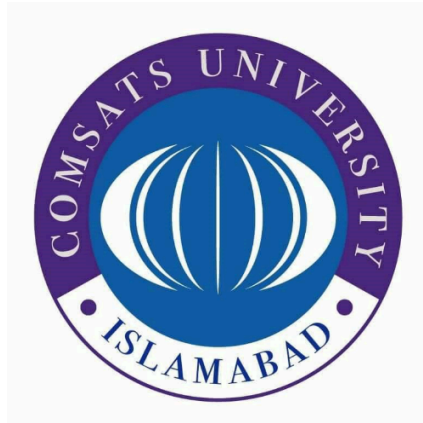




PARALLEL & DISTRIBUTED COMPUTING

LAB ASSIGNMENT #3

PREPARED BY

M Mukaram Awan
SP22-BCS-141

SUBMITTED TO

Sir Akhzar Nazeer

```

#include <iostream>
#include <cuda_runtime.h>

#define CHECK(call)
do {
    cudaError_t err = call;
    if (err != cudaSuccess) {
        std::cerr << "CUDA error: " << cudaGetErrorString(err)
            << " (" << __FILE__ << ":" << __LINE__ << ")\n";
        exit(EXIT_FAILURE);
    }
} while (0)

const int N = 1024;
const int THREADS = 256;
const int BLOCKS = (N + THREADS - 1) / THREADS;

// Kernel A: elementwise add C = A + B
__global__ void add_kernel(const int* A, const int* B, int* C, int n) {
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if (gid < n) C[gid] = A[gid] + B[gid];
}

// Kernel B: elementwise square D = C * C
__global__ void square_kernel(const int* C, int* D, int n) {
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if (gid < n) D[gid] = C[gid] * C[gid];
}

// Block-reduction kernel: each block reduces into shared memory; block result is added
// to global 'result' using atomicAdd (safe across blocks).
__global__ void reduce_block_shared(const int* D, int* result, int n) {
    extern __shared__ int sdata[]; // dynamic shared mem sized by caller
    int tid = threadIdx.x;
    int gid = blockIdx.x * blockDim.x + tid;

    // Load into shared memory (pad with 0 if out-of-range)
    sdata[tid] = (gid < n) ? D[gid] : 0;

```

```

__syncthreads();

// Classic tree-based reduction in shared memory
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
    if (tid < stride) sdata[tid] += sdata[tid + stride];
    __syncthreads();
}

// Thread 0 writes block sum to global using atomicAdd
if (tid == 0) atomicAdd(result, sdata[0]);
}

// Small helper kernel: fill an array with global thread ids (demonstrates indexing)
__global__ void fill_global_id(int* out, int n) {
    int gid = blockIdx.x * blockDim.x + threadIdx.x;
    if (gid < n) out[gid] = gid;
}

int main() {
    const int bytes = N * sizeof(int);

    // Host buffers
    int* h_A = new int[N];
    int* h_B = new int[N];
    int* h_C = new int[N];
    int* h_D = new int[N];
    long long h_sum_from_gpu = 0; // we'll copy int result into this after casting

    // Initialize as requested: A[i] = i, B[i] = 2*i
    for (int i = 0; i < N; ++i) {
        h_A[i] = i;
        h_B[i] = 2 * i;
    }

    // Device buffers
    int *d_A, *d_B, *d_C, *d_D, *d_result_int;
    CHECK(cudaMalloc((void**)&d_A, bytes));
    CHECK(cudaMalloc((void**)&d_B, bytes));

```

```

CHECK(cudaMalloc((void**)&d_C, bytes));
CHECK(cudaMalloc((void**)&d_D, bytes));
CHECK(cudaMalloc((void**)&d_result_int, sizeof(int)));

// Copy inputs to device
CHECK(cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice));
CHECK(cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice));

// ----- PART 1: Serial launches on default stream (stream 0) -----
std::cout << "=== SERIAL (default stream) run ===\n";
CHECK(cudaMemset(d_result_int, 0, sizeof(int)));

add_kernel<<<BLOCKS, THREADS>>>(d_A, d_B, d_C, N);
square_kernel<<<BLOCKS, THREADS>>>(d_C, d_D, N);

// If omitted, host may continue before GPU finishes -> potential race when copying results.
CHECK(cudaDeviceSynchronize());

// Reduction using shared-memory kernel
// shared memory size = THREADS * sizeof(int)
reduce_block_shared<<<BLOCKS, THREADS, THREADS * sizeof(int)>>>(d_D, d_result_int, N);
CHECK(cudaDeviceSynchronize());

// Copy results back
CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
CHECK(cudaMemcpy(h_D, d_D, bytes, cudaMemcpyDeviceToHost));
int h_result_int = 0;
CHECK(cudaMemcpy(&h_result_int, d_result_int, sizeof(int), cudaMemcpyDeviceToHost));
h_sum_from_gpu = (long long)h_result_int;

std::cout << "C (first 10): ";
for (int i = 0; i < 10; ++i) std::cout << h_C[i] << " ";
std::cout << "\nD (first 10): ";
for (int i = 0; i < 10; ++i) std::cout << h_D[i] << " ";
std::cout << "\nSum(D) from GPU (serial run): " << h_sum_from_gpu << "\n\n";

// Compute CPU reference sum to verify correctness (C = A+B, D = C*C)
long long cpu_sum = 0;

```

```

for (int i = 0; i < N; ++i) {
    long long c = (long long)h_A[i] + (long long)h_B[i];
    cpu_sum += c * c;
}
std::cout << "Reference CPU sum(D): " << cpu_sum << "\n\n";

// ----- PART 2: Streams + Events (attempt to overlap or to coordinate) -----
// Important: kernel2 depends on kernel1's output (C). Launching them in separate streams
// without synchronization causes a data hazard (kernel2 might read C before kernel1 writes
// it).
// We use an event to ensure kernel2 waits until kernel1 has finished computing C.
std::cout << "=== STREAMS with event synchronization ===\n";

// Reset device result and memory to show fresh run
CHECK(cudaMemset(d_result_int, 0, sizeof(int)));
CHECK(cudaMemset(d_C, 0xCD, bytes)); // fill with pattern (diagnostic), not required
CHECK(cudaMemset(d_D, 0xCD, bytes));

cudaStream_t s1, s2;
CHECK(cudaStreamCreate(&s1));
CHECK(cudaStreamCreate(&s2));
cudaEvent_t ev;
CHECK(cudaEventCreate(&ev));

// Launch add on stream s1
add_kernel<<<BLOCKS, THREADS, 0, s1>>>(d_A, d_B, d_C, N);

// Record event when stream s1's work up to this point is done
CHECK(cudaEventRecord(ev, s1));

// Make stream s2 wait for event before executing kernel2.
// This enforces: kernel2 (on s2) will start only after add_kernel finishes on s1.
CHECK(cudaStreamWaitEvent(s2, ev, 0));

// Launch square on stream s2 (will wait for the event). The kernels are on different streams
// but the event enforces correct ordering of the dependent data.
square_kernel<<<BLOCKS, THREADS, 0, s2>>>(d_C, d_D, N);

```

```

// Make sure all work completes
CHECK(cudaDeviceSynchronize());

// Reduction (on default stream) and copy back result
reduce_block_shared<<<BLOCKS, THREADS, THREADS * sizeof(int)>>>(d_D, d_result_int, N);
CHECK(cudaDeviceSynchronize());

CHECK(cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost));
CHECK(cudaMemcpy(h_D, d_D, bytes, cudaMemcpyDeviceToHost));
h_result_int = 0;
CHECK(cudaMemcpy(&h_result_int, d_result_int, sizeof(int), cudaMemcpyDeviceToHost));
std::cout << "C (first 10) after streams+event: ";
for (int i = 0; i < 10; ++i) std::cout << h_C[i] << " ";
std::cout << "\nD (first 10) after streams+event: ";
for (int i = 0; i < 10; ++i) std::cout << h_D[i] << " ";
std::cout << "\nSum(D) from GPU (streams+event): " << (long long)h_result_int << "\n\n";

// Clean up streams/events
CHECK(cudaEventDestroy(ev));
CHECK(cudaStreamDestroy(s1));
CHECK(cudaStreamDestroy(s2));

// ----- PART 3: Thread-hierarchy demonstration -----
// Launch add_kernel twice with different grid shapes and show how thread/block indices
map.
// We'll fill an array with global ids (gid = blockIdx.x*blockDim.x + threadIdx.x) and print a few
entries.

int *d_gid, *h_gid;
CHECK(cudaMalloc((void**)&d_gid, bytes));
h_gid = new int[N];

std::cout << "=== Thread-indexing demo: launch with <<<1, N>>> ===\n";
fill_global_id<<<1, N>>>(d_gid, N);
CHECK(cudaDeviceSynchronize());
CHECK(cudaMemcpy(h_gid, d_gid, bytes, cudaMemcpyDeviceToHost));
std::cout << "First 10 'global ids' (1 block, N threads): ";
for (int i = 0; i < 10; ++i) std::cout << h_gid[i] << " ";

```

```

std::cout << "\n(Here threadIdx runs 0.." << (N-1) << ", blockIdx == 0)\n\n";

std::cout << "=== Thread-indexing demo: launch with <<<N/32, 32>>> ===\n";
int blocks32 = N / 32; // 1024/32 = 32
fill_global_id<<<blocks32, 32>>>(d_gid, N);
CHECK(cudaDeviceSynchronize());
CHECK(cudaMemcpy(h_gid, d_gid, bytes, cudaMemcpyDeviceToHost));
std::cout << "First 10 'global ids' (N/32 blocks, 32 threads): ";
for (int i = 0; i < 10; ++i) std::cout << h_gid[i] << " ";
std::cout << "\n(Here threadIdx runs 0..31 within each block; blockIdx varies 0.." <<
(blocks32-1) << ")\n\n";

// ----- Final housekeeping -----
delete[] h_gid;
CHECK(cudaFree(d_gid));

CHECK(cudaFree(d_A));
CHECK(cudaFree(d_B));
CHECK(cudaFree(d_C));
CHECK(cudaFree(d_D));
CHECK(cudaFree(d_result_int));

delete[] h_A;
delete[] h_B;
delete[] h_C;
delete[] h_D;

std::cout << "Done.\n";
return 0;
}

```